Practical 11

Implementation of Floyd–Warshall algorithm (All-Pairs Shortest Path)

1. Introduction

The Floyd–Warshall algorithm is a classic dynamic programming algorithm used to find the shortest paths between **all pairs of vertices** in a weighted directed graph. Unlike Dijkstra's algorithm, which finds shortest paths from a single source, Floyd–Warshall computes the shortest distance between every pair of vertices in a single run.

The algorithm works by gradually allowing more and more vertices to act as intermediate vertices. For every vertex k, it checks whether the path from i to j through k is shorter than the best path found so far. It can handle negative edge weights, provided the graph has no negative cycles.

2. Aim

To implement the Floyd–Warshall algorithm in C++ and determine the shortest distance between every pair of vertices in a given weighted directed graph.

3. Objectives

- To understand the concept of the all-pairs shortest path problem.
- To understand the Dynamic Programming approach used in Floyd–Warshall.
- To compute the shortest distance matrix for a weighted graph.
- To handle unreachable vertices using an INF (infinity) value.
- To analyze the time and space complexity of the algorithm.
- To implement the algorithm using C++.

4. Algorithm

Step 1: Start.

Step 2: Read the number of vertices V and the adjacency matrix graph[V][V], where graph[i][j] is the weight of the edge from i to j (0 if i = j, INF if no edge exists).

Step 3: Create a distance matrix dist[V][V] and copy all values of graph into it.

Step 4: For each vertex k from 0 to V–1 (intermediate vertex):

- For each vertex i from 0 to V–1 (source vertex).
- For each vertex j from 0 to V–1 (destination vertex).

Step 5: If dist[i][k] and dist[k][j] are both not INF, calculate dist[i][k] + dist[k][j].

Step 6: If this sum is less than dist[i][j], update dist[i][j] with the sum.

Step 7: Repeat Steps 4–6 until all vertices have been used as intermediate vertices.

Step 8: The matrix dist[V][V] now holds the shortest distance between every pair of vertices.

Step 9: Print the matrix, displaying INF where no path exists.

Step 10: Stop.

5. Pseudocode

```
BEGIN
FUNCTION FloydWarshall(graph, V)
   Create distance matrix dist[V][V]
   FOR i = 0 TO V-1
      FOR j = 0 TO V-1
         dist[i][j] = graph[i][j]
      END FOR
   END FOR

   FOR k = 0 TO V-1
      FOR i = 0 TO V-1
         FOR j = 0 TO V-1
            IF dist[i][k] != INF AND dist[k][j] != INF THEN
               IF dist[i][k] + dist[k][j] < dist[i][j] THEN
                  dist[i][j] = dist[i][k] + dist[k][j]
               END IF
            END IF
         END FOR
      END FOR
   END FOR

   RETURN dist
END FUNCTION
END
```

Code:

```
#include <iostream>
using namespace std;

#define V 4
#define INF 99999

void floydWarshall(int graph[V][V]) {
    int dist[V][V];

    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
```

```
            dist[i][j] = graph[i][j];

    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] != INF &&
                    dist[k][j] != INF &&
                    dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    cout << "Shortest distance matrix:\n";

    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                cout << "INF ";
            else
                cout << dist[i][j] << " ";
        }
        cout << endl;
    }
}

int main() {
    int graph[V][V] = {
        {0,   5,   INF, 10},
        {INF, 0,   3,   INF},
        {INF, INF, 0,   1},
        {INF, INF, INF, 0}
    };

    floydWarshall(graph);

    return 0;
}
```

Input Graph

The program uses a directed graph with 4 vertices (0 to 3) and the following edges: 0→1 (5), 0→3 (10), 1→2 (3), 2→3 (1). Missing edges are represented by INF (99999).

From \ To	0	1	2	3
0	0	5	INF	10
1	INF	0	3	INF
2	INF	INF	0	1
3	INF	INF	INF	0

Output

```
Shortest distance matrix:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0
```

Final shortest distance matrix:

From \ To	0	1	2	3
0	0	5	8	9
1	INF	0	3	4
2	INF	INF	0	1
3	INF	INF	INF	0

Explanation: the shortest path from 0 to 2 is 0→1→2 with cost 5 + 3 = 8. The shortest path from 0 to 3 is 0→1→2→3 with cost 5 + 3 + 1 = 9, which is shorter than the direct edge 0→3 of cost 10. Vertices 1, 2 and 3 cannot reach vertex 0, so those entries remain INF.

6. Advantages

- Finds the shortest paths between all pairs of vertices in one execution.
- Simple and easy to implement using three nested loops.
- Works correctly with negative edge weights (if there is no negative cycle).
- Works for both dense and sparse graphs and can detect negative cycles.
- Has polynomial time complexity of $O(V^3)$.

7. Disadvantages

- Requires $O(V^2)$ memory to store the distance matrix.
- Time complexity $O(V^3)$ becomes inefficient for graphs with a very large number of vertices.
- For a single-source shortest path problem, Dijkstra's or Bellman–Ford is more efficient.
- Does not work correctly if the graph contains a negative cycle.
- The INF value must be chosen carefully to avoid overflow during addition.

8. Complexity Analysis

Complexity	**Value**
Time Complexity	$O(V^3)$
Space Complexity	$O(V^2)$

Where:

- V = number of vertices in the graph

9. Conclusion

The Floyd–Warshall algorithm was successfully implemented in C++. The algorithm uses dynamic programming and considers every vertex as a possible intermediate vertex to improve the shortest distance between each pair of vertices. The program produced the correct shortest distance matrix for the given graph, with INF shown for unreachable pairs. Thus, Floyd–Warshall provides a simple and systematic approach for solving the all-pairs shortest path problem.